Functions & Scope

Bit by Bit Coding - Term 2 | Week 2

Key Concepts

- A function is a reusable block of code: def name(params):.
- Parameters are inputs; return sends a value back to the caller.
- Scope: local variables live inside a function; global live outside.
- Default arguments give a parameter a fallback value.
- *args collects extra positional args; **kwargs collects keyword args.
- lambda creates a small one-line function.

Syntax Reference

Defining & Calling

```
def greet(name):
    return "Hello, " + name

msg = greet("Sara")
print(msg)          # Hello, Sara
```

Parameters & Defaults

```
def power(base, exp=2):       # exp defaults to 2
    return base ** exp
print(power(5))             # 25
print(power(2, 3))          # 8
```

*args & **kwargs

```
def total(*args):
    return sum(args)
print(total(1, 2, 3, 4))    # 10

def info(**kwargs):
    for k, v in kwargs.items():
        print(k, v)
info(name="Ali", age=15)
```

Return & Scope

```
counter = 0           # global
def bump():
    global counter
    counter += 1
    return counter
print(bump())        # 1
```

Lambda Functions

```
square = lambda x: x * x
print(square(6))     # 36
nums = [3, 1, 2]
nums.sort(key=lambda n: n)   # sort by value
```

Common Patterns

- Functions that return values are easier to test than ones that just print.
- Use return to send data back; use print only to display.
- Pass functions as arguments (e.g., key= to sorted/sort).

Tips & Common Mistakes

- Forgetting return: a function with no return gives None.
- Using a global inside a function needs the global keyword.
- Define the function before you call it.
- Parameters are local - changing them won't affect the caller's variables.